

Technical Analysis of a Fully Connected Neural Network for Persian Handwritten Digit Recognition

HODA Dataset, Reproducible Preprocessing, Optimizer Selection, and
TensorFlow/Keras Implementation

Publication-Oriented Technical Report

Contents

Abstract	2
1 Introduction	2
2 Project Overview	3
3 Technical Foundations	3
3.1 Supervised multiclass classification	3
3.2 Dense neural transformations	3
3.3 Batch normalization	4
3.4 Dropout	4
3.5 Softmax and cross-entropy	4
4 Data and Input Processing	4
4.1 Input contract	4
4.2 Exact reconstruction of variable-sized images	5
4.3 Resizing and intensity scaling	5
4.4 Flattening	5
4.5 Validation split and feature standardization	6
5 System Architecture and Code Organization	6
6 Detailed Methodology	7
6.1 Environment and reproducibility controls	7
6.2 Data loading and defensive validation	7
6.3 Model construction	7
6.4 Optimizer abstraction	7
6.5 Controlled hyperparameter search	8
6.6 Final training procedure	8
6.7 Test evaluation	9
6.8 Prediction and diagnostics	9
7 Mathematical Formulation	9
7.1 End-to-end mapping	9
7.2 Hidden-layer transformations	9
7.3 Accuracy and error rate	10

7.4	Confusion matrix	10
8	Optimization and Learning Procedure	10
9	Implementation Details	10
9.1	Library responsibilities	10
9.2	Data types	11
9.3	Batch size	11
9.4	Runtime artifacts	11
10	Execution Workflow	11
11	Design Decisions and Rationale	12
12	Computational Considerations	12
12.1	Parameter and forward-pass cost	12
12.2	Memory behavior	12
12.3	Potential bottlenecks	12
13	Reproducibility and Reliability	13
14	Limitations and Technical Considerations	13
14.1	Representation limits	13
14.2	Resolution trade-off	13
14.3	Search-space limitations	13
14.4	Validation protocol nuance	13
14.5	Dependency and environment constraints	14
15	Overall Technical Assessment	14
16	Conclusion	14
	Source-to-Report Traceability Note	15

Abstract

This report analyzes a Python implementation for the second assignment of a deep-learning course on Persian handwritten digit recognition with a fully connected neural network. The implementation reconstructs variable-sized HODA images from compact CSV representations, validates the dataset contract, resizes each image to 20×20 pixels with anti-aliasing, scales intensities to $[0, 1]$, flattens the images to 400-dimensional vectors, and applies feature standardization based only on the training fit partition. The classifier is a sequential dense network with the architecture $400 \rightarrow 256 \rightarrow 128 \rightarrow 10$, ReLU activations, batch normalization, dropout, and a softmax output layer. Rather than fixing the optimizer in advance, the code compares Adam, RMSprop, and momentum-based Nesterov SGD at two learning rates each and selects the configuration with the highest validation accuracy. A final model is then trained from scratch with early stopping and learning-rate reduction, evaluated with Keras `evaluate`, and used to generate class-probability predictions for the test set. The implementation also produces confusion-matrix and per-class diagnostics and packages the runtime artifacts into a ZIP archive.

The report distinguishes documented intent from implementation behavior. In particular, the phrase that the final stage “uses all 10,000 training samples” is qualified by the actual `validation_split=0.15` call: the model receives the complete 10,000-example array, but 15% is held out from gradient updates during the final training run. No numerical performance results are asserted here because the uploaded source computes those quantities at execution time rather than embedding fixed outcomes. The analysis therefore concentrates on architecture, mathematical formulation, data flow, algorithmic behavior, reproducibility, computational cost, and implementation limitations.

1 Introduction

The uploaded source is a self-contained Python implementation of a deep-learning assignment for classifying Persian handwritten digits. Its stated requirements are unusually useful for understanding the intended workflow: images must be represented at 20×20 resolution; a neural network must be designed and trained; the training process must be inspected; the trained network must be evaluated on a held-out test set; incorrect predictions must be analyzed; the probability vector for the first test image must be examined; and several optimizer/learning-rate combinations must be compared (source lines 16–25).

The code therefore serves two roles. Computationally, it is a supervised multiclass classification pipeline. As an experiment script, it also fixes random seeds, validates the input contract, records hyperparameter-search outcomes, saves the trained model, exports diagnostics, writes machine-readable summaries, and packages the output directory. Together, these features provide a reproducible execution path from serialized images to a trained classifier and a set of auditable artifacts.

The report focuses on what the source actually implements. It does not reconstruct a missing MATLAB implementation, and it does not introduce performance numbers not present in the uploaded file. The source states that the original archive included a PDF specification, HODA data, and a starter notebook, while no MATLAB files were available; the Python notebook was therefore treated as the executable reference implementation (source lines 12–14).

2 Project Overview

At a high level, the project follows this computational pipeline:

1. initialize the software environment and reproducibility controls;
2. establish a project-relative directory structure;
3. locate or upload the training and test CSV files;
4. validate row counts, split labels, class labels, image dimensions, and pixel-vector integrity;
5. decode each row into an exact variable-sized grayscale image;
6. resize every image to 20×20 pixels with anti-aliasing and scale intensities to $[0, 1]$;
7. flatten the images to 400-dimensional feature vectors;
8. create a reproducible, stratified validation split for the hyperparameter search;
9. estimate feature-wise standardization statistics from the fit partition only;
10. compare six optimizer/learning-rate configurations under a fixed architecture and training budget;
11. select the best configuration by validation accuracy with validation loss as a tie-breaking criterion;
12. construct a fresh final model and train it with early stopping and adaptive learning-rate reduction;
13. evaluate the final network on the untouched 2,000-example test set;
14. predict class probabilities, identify misclassifications, compute a confusion matrix and classification report, and inspect the first test sample's probability vector;
15. write structured summaries, save the model, and package the complete results directory.

This sequence is visible in the source code and comments: data loading and validation at lines 89–142, preprocessing at lines 144–226, architecture and optimizer search at lines 228–351, final training and evaluation at lines 353–427, diagnostics at lines 428–505, and packaging at lines 543–569.

3 Technical Foundations

3.1 Supervised multiclass classification

Let a preprocessed input image be represented by a vector $\mathbf{x} \in \mathbb{R}^{400}$ and its digit label by $y \in \{0, 1, \dots, 9\}$. The network learns a parameterized mapping

$$\hat{\mathbf{p}} = f_{\theta}(\mathbf{x}), \quad \hat{\mathbf{p}} \in [0, 1]^{10}, \quad \sum_{k=0}^9 \hat{p}_k = 1, \quad (1)$$

where θ denotes all trainable parameters. The predicted class is

$$\hat{y} = \arg \max_{k \in \{0, \dots, 9\}} \hat{p}_k. \quad (2)$$

The code obtains $\hat{y}$ by applying `numpy.argmax` along the class dimension of the matrix returned by `model.predict`.

3.2 Dense neural transformations

For a dense layer with input $\mathbf{h}^{(\ell-1)}$, weights $W^{(\ell)}$, and bias $\mathbf{b}^{(\ell)}$, the affine transformation is

$$\mathbf{z}^{(\ell)} = W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}. \quad (3)$$

The hidden layers use the rectified linear unit,

$$\text{ReLU}(z) = \max(0, z), \quad (4)$$

which provides the nonlinear capacity needed to represent nonlinear decision boundaries.

3.3 Batch normalization

The code inserts batch normalization before the ReLU activation in both hidden blocks. In simplified training-time form, a batch-normalized feature can be written as

$$\text{BN}(z) = \gamma \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta, \quad (5)$$

where μ_B and σ_B^2 are batch statistics, γ and β are learned scale and offset parameters, and ϵ is a numerical-stability constant supplied internally by Keras. The exact internal details are delegated to TensorFlow/Keras; the source selects the layer but does not override its normalization hyperparameters.

3.4 Dropout

Dropout is used after each hidden activation, with rates 0.25 and 0.20. Conceptually, training-time dropout multiplies activations by random Bernoulli masks and rescales the survivors. Its role is regularization rather than feature extraction. The source does not implement dropout manually; it delegates behavior to Keras.

3.5 Softmax and cross-entropy

The final dense layer has ten outputs and uses softmax:

$$\hat{p}_k = \frac{\exp(z_k)}{\sum_{j=0}^9 \exp(z_j)}. \quad (6)$$

Because labels are integer class IDs rather than one-hot vectors, the implementation selects `sparse_categorical_crossentropy`. For a single observation with true class y , the loss can be expressed as

$$\mathcal{L}(\theta; \mathbf{x}, y) = -\log \hat{p}_y. \quad (7)$$

For N observations, the empirical objective is

$$\mathcal{J}(\theta) = \frac{1}{N} \sum_{i=1}^N -\log p_\theta(y_i \mid \mathbf{x}_i). \quad (8)$$

The source reports accuracy as an additional metric but does not optimize accuracy directly.

4 Data and Input Processing

4.1 Input contract

The model does not read the original MATLAB `.mat` file. The executable path consumes two CSV files, `hoda_train.csv` and `hoda_test.csv`, stored under `datasets/`. Each row is

expected to contain six fields: `sample_id`, `split`, `label`, `height`, `width`, and a space-separated `pixels` string. The source explicitly requires 10,000 training rows and 2,000 test rows.

The `validate_dataset` function checks four properties. First, the row count must match the expected split size. Second, the `split` column must contain exactly the expected string. Third, labels must lie in the inclusive range 0 through 9. Fourth, image dimensions must be positive and pixel strings must not be missing (source lines 124–136). A separate reconstruction-time check later verifies that the number of decoded pixel values equals `height  $\times$  width` for every sample.

This validation strategy is valuable because the model otherwise has no way to distinguish a malformed serialization from a valid image. The source fails early with explicit exceptions rather than allowing corrupted rows to propagate into TensorFlow.

4.2 Exact reconstruction of variable-sized images

The function `decode_csv_images` converts each compact row into a two-dimensional NumPy array. The pixel string is parsed into unsigned 8-bit integers and reshaped according to the recorded height and width. Thus, before resizing, the representation remains faithful to the dimensions declared by the dataset metadata.

If sample i has dimensions $H_i \times W_i$, the reconstruction yields

$$X_i^{\text{orig}} \in \{0, \dots, 255\}^{H_i \times W_i}. \quad (9)$$

The variable image size is resolved only at the next stage; CSV decoding itself does not impose an artificial shape.

4.3 Resizing and intensity scaling

Every original image is resized to 20×20 using `skimage.transform.resize` with `preserve_range=True` and `anti_aliasing=True`.

Anti-aliasing is a reasonable choice for downsampling because it reduces high-frequency artifacts that can arise when variable-sized images are mapped to a common resolution.

The resulting floating-point image is divided by 255:

$$X_i^{(20)} = \frac{1}{255} \text{Resize}_{20 \times 20}(X_i^{\text{orig}}), \quad (10)$$

so the feature values are nominally in $[0, 1]$, subject to the numerical behavior of the interpolation routine. The explicit `preserve_range=True` flag prevents the resize function from reinterpreting the original intensity range before the explicit division by 255.

4.4 Flattening

The 20×20 array is reshaped to a length-400 vector:

$$\mathbf{x}_i = \text{vec}(X_i^{(20)}) \in \mathbb{R}^{400}. \quad (11)$$

From the network’s perspective, this operation removes the two-dimensional tensor structure. Pixel adjacency is represented only through the ordering of the flattened vector; the architecture itself has no convolutional or spatially local operators.

4.5 Validation split and feature standardization

The hyperparameter search uses `train\_test\_split` with `test\_size=0.15`, `random\_state=42`, and `stratify=y_train`. Thus, the 10,000 training examples are partitioned into an 8,500-example fit subset and a 1,500-example validation subset while approximately preserving class proportions.

Feature-wise standardization is then computed only from the fit subset:

$$\mu_j = \frac{1}{N_{\text{fit}}} \sum_{i=1}^{N_{\text{fit}}} x_{ij}, \quad \sigma_j = \sqrt{\frac{1}{N_{\text{fit}}} \sum_{i=1}^{N_{\text{fit}}} (x_{ij} - \mu_j)^2}. \quad (12)$$

The source replaces any standard deviation below 10^{-6} with 1.0 to avoid unstable division. Each feature is then transformed as

$$z_{ij} = \frac{x_{ij} - \mu_j}{\tilde{\sigma}_j}. \quad (13)$$

The same μ_j and $\tilde{\sigma}_j$ are applied to the validation, complete-training, and test arrays. This prevents validation and test statistics from influencing the standardization parameters.

5 System Architecture and Code Organization

The source is a single executable Python module organized as a notebook-style script. It defines six helper functions followed by a sequence of top-level execution stages. This separation is sufficient for an assignment-sized project while keeping the end-to-end flow visible.

Component	Source lines	Responsibility
<code>validate_dataset</code>	124–134	Check split size, split label, digit range, dimensions, and missing pixel vectors.
<code>decode_csv_images</code>	151–163	Parse row-major pixel strings and reconstruct exact 2-D images.
<code>resize_to_20x20</code>	165–175	Resize variable-sized images and scale intensities.
<code>build_model</code>	241–257	Construct the dense $400 \rightarrow 256 \rightarrow 128 \rightarrow 10$ network.
<code>make_optimizer</code>	259–267	Instantiate Adam, RMSprop, or Nesterov momentum SGD.
<code>compile_model</code>	269–275	Attach the selected optimizer, sparse cross-entropy loss, and accuracy metric.

The top-level stages are also clear. Environment setup imports libraries and controls randomness; filesystem setup selects either `/content` or the current working directory; data-loading logic creates the CSV contract; preprocessing transforms the data; model code defines the hypothesis class and optimization alternatives; training and evaluation execute the experiment; diagnostics explain failure modes; and packaging converts the output directory into a single archive.

Two imports, `os` and `math`, are present but are not materially used in the shown implementation. This does not affect correctness, but the unused imports add minor noise to the import section.

6 Detailed Methodology

6.1 Environment and reproducibility controls

The implementation attempts to import TensorFlow and installs a version in the interval $[2.15, 2.21)$ only if the import fails. It then seeds Python’s `random` module, NumPy, and TensorFlow with the fixed value 42. The code also reports library versions and available GPU devices.

These controls improve reproducibility but do not guarantee bitwise determinism. The source does not enable TensorFlow deterministic operations explicitly, and GPU execution can contain operations whose ordering or kernel implementation varies across environments. A more accurate description is therefore “controlled stochastic initialization and sampling” rather than absolute determinism.

6.2 Data loading and defensive validation

The notebook checks for the existence of both CSV files. In Colab, missing files trigger an interactive upload. Outside Colab, the code expects the files to exist under the configured `datasets/` directory and raises a `FileNotFoundError` otherwise.

Using project-relative directories is a practical reproducibility choice. The source distinguishes Colab execution using `"google.colab"` in `sys.modules` and otherwise falls back to `Path.cwd()`. The same code can therefore run in both environments without hard-coding every output path.

6.3 Model construction

The core model is

```
Input(400)
Dense(256) -> BatchNormalization -> ReLU -> Dropout(0.25)
Dense(128) -> BatchNormalization -> ReLU -> Dropout(0.20)
Dense(10, activation='softmax')
```

The architecture contains 137,610 trainable parameters and 768 non-trainable batch-normalization statistics, for 138,378 parameters in total. These counts follow directly from the layer dimensions and Keras’ batch-normalization parameterization. The first dense layer contributes $400 \times 256 + 256 = 102,656$ trainable parameters; the first batch-normalization layer contributes $2 \times 256 = 512$ trainable parameters; the second dense layer contributes $256 \times 128 + 128 = 32,896$; the second batch-normalization layer contributes $2 \times 128 = 256$; and the output layer contributes $128 \times 10 + 10 = 1,290$.

The network is intentionally modest in size. It is large enough to learn nonlinear class boundaries in a 400-dimensional input space while remaining small enough for rapid experimentation on the assignment-scale dataset.

6.4 Optimizer abstraction

The `make_optimizer` function isolates optimizer selection from model construction. Three choices are supported:

- Adam with the requested learning rate;
- RMSprop with the requested learning rate;
- SGD with momentum 0.9 and Nesterov acceleration, using the requested learning rate.

This design is preferable to embedding optimizer logic directly into the training loop because the search loop can vary the optimization method without duplicating model-construction code.

6.5 Controlled hyperparameter search

Six configurations are evaluated: Adam at 10^{-3} and 3×10^{-4} ; RMSprop at 10^{-3} and 3×10^{-4} ; and Nesterov SGD at 10^{-2} and 10^{-3} . The architecture, batch size, validation split, seed, and search epoch budget remain fixed.

For each configuration, the session is cleared, random seeds are reset, a fresh model is created, and `fit` is called for 12 epochs. The ranking criterion is lexicographic: maximize the best validation accuracy, then minimize the best validation loss. Formally, if a configuration c has validation sequence $a_{c,1}, \dots, a_{c,T}$ and loss sequence $\ell_{c,1}, \dots, \ell_{c,T}$, the stored quantities are

$$A_c = \max_t a_{c,t}, \quad L_c = \min_t \ell_{c,t}. \quad (14)$$

The selected configuration maximizes A_c and uses L_c as the tie-breaker.

This is a reasonable small-scale experimental design because it isolates two high-impact optimizer hyperparameters while holding the hypothesis class constant. It is not a full hyperparameter optimization procedure: only six manually specified settings are tested, and the search budget is relatively short. The selected configuration should therefore be interpreted as the best observed among the tested alternatives, not as a globally optimal choice.

6.6 Final training procedure

The chosen optimizer and learning rate are used to create a new model. This avoids carrying a search model forward as the final checkpoint; the final model starts from a new initialization and is trained again.

Two callbacks control the final training dynamics. `EarlyStopping` monitors validation loss, waits five epochs without improvement, and restores the best observed weights. `ReduceLROnPlateau` halves the learning rate after two stagnant validation-loss epochs, subject to a floor of 10^{-5} .

The final call uses `validation\_split=0.15`. Therefore, although the full 10,000-example training array is supplied to `fit`, only 85% of those examples contribute to gradient updates in a given training epoch; the remaining 15% serve as validation data. This implementation detail should be stated explicitly when interpreting the notebook’s phrase “uses all 10,000 training samples.”

A second nuance is that the final validation subset is not created with the earlier explicit stratified splitter. Keras selects its validation portion internally from the array supplied to `fit`. The source does not explicitly request stratification at this stage. This is not necessarily incorrect, but it means the final validation split is controlled differently from the search split.

6.7 Test evaluation

The test set is held until after final model selection and training. The call to `evaluate` returns the final test loss and accuracy. The test set therefore serves as an external evaluation set rather than a tuning signal.

6.8 Prediction and diagnostics

The source obtains a full test probability matrix with `model.predict`. For each sample, the predicted digit is the index of the largest probability. Misclassified indices are extracted by comparing predictions with the true labels.

Additional diagnostics include a confusion matrix and a scikit-learn classification report. These diagnostics are useful because a single aggregate accuracy can hide asymmetric class performance. The confusion matrix counts the number of examples whose true class is i and predicted class is j , while the classification report provides class-wise precision, recall, and F1-related summaries.

The first test sample receives special treatment because the assignment explicitly requires its complete probability vector. The code stores the vector, the predicted digit, the true digit, and the correctness flag, and then exports the probability values to CSV.

7 Mathematical Formulation

7.1 End-to-end mapping

For each original image I_i of shape $H_i \times W_i$, preprocessing can be expressed as

$$I_i^{20} = \text{Resize}_{20 \times 20}(I_i), \quad (15)$$

$$X_i = \text{vec} \left(\frac{I_i^{20}}{255} \right), \quad (16)$$

followed by training-derived standardization,

$$Z_{ij} = \frac{X_{ij} - \mu_j}{\tilde{\sigma}_j}. \quad (17)$$

The resulting vector $\mathbf{z}_i \in \mathbb{R}^{400}$ is passed to the network.

7.2 Hidden-layer transformations

The first hidden block can be represented as

$$\mathbf{u}^{(1)} = W^{(1)}\mathbf{z} + \mathbf{b}^{(1)}, \quad (18)$$

$$\mathbf{v}^{(1)} = \text{BN}(\mathbf{u}^{(1)}), \quad (19)$$

$$\mathbf{h}^{(1)} = \text{ReLU}(\mathbf{v}^{(1)}), \quad (20)$$

$$\tilde{\mathbf{h}}^{(1)} = \text{Dropout}_{0.25}(\mathbf{h}^{(1)}). \quad (21)$$

The second block follows the same pattern with 128 hidden units and dropout rate 0.20. The final logits are

$$\mathbf{o} = W^{(3)}\tilde{\mathbf{h}}^{(2)} + \mathbf{b}^{(3)}, \quad (22)$$

and softmax converts them to class probabilities.

7.3 Accuracy and error rate

The classification accuracy used by Keras can be expressed as

$$\text{Acc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i]. \quad (23)$$

The notebook defines the requested classification error curve as

$$\text{Err} = 1 - \text{Acc}. \quad (24)$$

This distinction matters because the training objective is cross-entropy, whereas the assignment visualizes classification error rate.

7.4 Confusion matrix

For $K = 10$ classes, the confusion matrix $C \in \mathbb{N}^{10 \times 10}$ is defined by

$$C_{ij} = \sum_{n=1}^N \mathbf{1}[y_n = i \wedge \hat{y}_n = j]. \quad (25)$$

Rows therefore correspond to true classes and columns to predicted classes, matching the labels in the source.

8 Optimization and Learning Procedure

The training objective is sparse multiclass cross-entropy. Gradient-based optimization updates the model parameters according to the optimizer-specific rule. The source delegates the detailed update equations to TensorFlow/Keras, but the model-level objective is clear.

For the six search configurations, the optimization budget is intentionally controlled. Each candidate receives the same 12-epoch budget and the same batch size of 256. This improves the interpretability of the comparison because the model architecture and basic schedule are not changing simultaneously with the optimizer.

The final training stage introduces adaptive control of the learning rate and an early stopping criterion based on validation loss. In effect, the learning schedule becomes piecewise adaptive: plateau detection reduces the step size, while early stopping terminates training when further validation-loss improvement has stalled long enough. This is a practical compromise between fixed training duration and uncontrolled over-training.

The source does not implement explicit L_1 or L_2 weight regularization, data augmentation, label smoothing, class weighting, or learning-rate warmup. Regularization is instead provided mainly by dropout and batch normalization, with validation monitoring and early stopping helping control training.

9 Implementation Details

9.1 Library responsibilities

The implementation uses NumPy for numerical arrays and vectorized preprocessing, pandas for tabular CSV handling and report creation, scikit-learn for the stratified split and evaluation

diagnostics, scikit-image for image resizing, and TensorFlow/Keras for the neural network and optimization procedure. Matplotlib is used for the notebook's visual diagnostics. Standard-library modules provide filesystem, JSON, subprocess, randomness, and archive operations.

9.2 Data types

The decoded source images are represented as unsigned 8-bit integers, preserving the compact pixel representation. Resized images are converted to 32-bit floating point values. The flattened standardized matrices are also stored as 32-bit floating point arrays. This is a reasonable balance between numerical precision and memory consumption for a 10,000-sample assignment.

9.3 Batch size

A batch size of 256 is used in both the optimizer search and final training, and the same batch size is passed to test evaluation and prediction. For a 400-feature input, the raw feature payload for one batch is modest; the main memory footprint comes from intermediate network activations and TensorFlow bookkeeping rather than from the input matrix alone.

9.4 Runtime artifacts

The source saves more than a single model file. It writes optimizer-search results, a trained Keras model, a classification report, first-image probabilities, a JSON summary, a run manifest, and visual diagnostic files under `results/`. The final archive is created with `shutil.make_archive`. In Colab, the ZIP file is then offered through `files.download`.

10 Execution Workflow

The complete execution sequence can be read as a dependency graph:

1. configuration creates directories and paths;
2. CSV existence checks establish whether the pipeline can proceed;
3. CSV parsing produces two data frames;
4. validation confirms the data contract;
5. decoding creates lists of original images and label arrays;
6. resizing and flattening create the fixed-dimensional feature matrices;
7. the stratified split defines fit and search-validation subsets;
8. the fit subset defines standardization statistics;
9. the standardized fit and validation arrays feed the six-model search;
10. the search table determines one optimizer-learning-rate pair;
11. a fresh model is compiled with that pair;
12. final training writes a model and training history;
13. test evaluation and prediction create final metrics and diagnostics;
14. JSON/CSV/model/image artifacts are collected in `results/`;
15. the results directory is compressed into one archive.

This ordering is important. For example, standardization must occur after the training/validation split if leakage is to be avoided, and the test set must not participate in optimizer selection. The code follows those dependencies correctly.

11 Design Decisions and Rationale

Several choices are particularly significant.

First, the 20-by-20 representation is imposed before flattening, satisfying the assignment while creating a stable 400-dimensional input contract. Second, anti-aliased resizing reduces the risk of severe aliasing when original HODA images have different dimensions. Third, standardization is learned from a non-test partition, preserving the conceptual separation between model development and final evaluation.

Fourth, the optimizer abstraction permits a controlled comparison without changing the network definition. Fifth, the fresh final model avoids carrying one search model forward as an implicit checkpoint. Sixth, the final callbacks use validation loss rather than training loss, which is aligned with the goal of selecting a generalizable stopping point.

The code does not state the author’s motivation for every design choice, so the preceding rationale should be read as an implementation-based interpretation, not as a claim about undocumented intent.

12 Computational Considerations

12.1 Parameter and forward-pass cost

The network has 138,378 total parameters, of which 137,610 are trainable. For one input vector, the dominant matrix multiplications are approximately

$$400 \times 256 + 256 \times 128 + 128 \times 10 = 136,448 \quad (26)$$

scalar multiply-add terms before accounting for bias, normalization, activation, and dropout operations. Thus the forward-pass cost per example is on the order of $O(400 \cdot 256 + 256 \cdot 128 + 128 \cdot 10)$.

For a training dataset with N examples, one complete epoch has computational work proportional to $O(N)$ for fixed architecture and batch size. The preprocessing cost is separate and is dominated by decoding and image-resizing work over the total number of input pixels.

12.2 Memory behavior

The fixed 20×20 representation makes the in-memory model input predictable. The standardized training matrix contains $10,000 \times 400$ floating-point values, which is approximately 4 million values. At 32 bits per value, that matrix alone is about 16 MB, excluding NumPy array overhead and additional copies. The code also retains the original variable-sized image lists, the resized 3-D arrays, validation arrays, and test arrays, so peak memory use is higher than this single-matrix estimate.

12.3 Potential bottlenecks

The image-resizing loop is written in Python and calls the resizing routine once per image. This is acceptable at the assignment scale, but it is one of the clearest CPU-side preprocessing costs. Training is more efficiently handled by TensorFlow and can exploit a GPU when one is

available. The code reports detected GPU devices but does not explicitly configure a device strategy, so device placement is left to TensorFlow.

13 Reproducibility and Reliability

The implementation makes several concrete reproducibility-oriented choices: fixed random seeds, explicit train/test file naming, deterministic directory creation, explicit expected dataset sizes, controlled validation splitting for the search, machine-readable JSON summaries, CSV exports, and saving the final Keras model.

Reproducibility is therefore controlled but not absolute. Exact results can still depend on TensorFlow version, hardware, GPU kernels, and floating-point execution. The source records the TensorFlow version in the run manifest, which is useful for later audit. It does not pin every dependency version, so the environment is not fully frozen.

The main reliability strength is the early validation of the serialized dataset. The main limitation is that the final validation split is controlled by Keras' internal `validation_split` mechanism rather than by the explicit stratified splitter used during optimizer search.

14 Limitations and Technical Considerations

14.1 Representation limits

The network receives flattened pixels and therefore does not exploit convolutional locality, translation-aware kernels, or pooling. For handwritten images, those spatial inductive biases can be important. The source itself acknowledges that a fully connected representation has limitations relative to convolutional architectures, particularly for modeling local structure.

14.2 Resolution trade-off

Resizing every sample to 20×20 creates a consistent input size but can remove fine stroke information. Anti-aliasing can reduce interpolation artifacts but cannot recover detail lost during downsampling. Consequently, some classification errors can plausibly be related to the chosen resolution, although the source correctly treats such explanations as hypotheses rather than universal facts.

14.3 Search-space limitations

The optimizer study examines only six manually selected configurations. It varies optimizer and learning rate but not width, dropout rates, batch size, image preprocessing parameters, or other architectural choices. The resulting “best” configuration is best only within the tested set.

14.4 Validation protocol nuance

The search stage uses a reproducible stratified holdout, whereas the final training call uses `validation_split=0.15`. This creates two slightly different validation protocols. The differ-

ence is not a critical methodological problem, but it matters when comparing search-validation behavior with the final training history.

14.5 Dependency and environment constraints

The TensorFlow installation path is opportunistic: it installs a compatible range only when TensorFlow cannot be imported. This is practical for Colab but does not guarantee an identical environment on every machine. For a production-grade research repository, a lockfile or exact environment specification would improve portability.

15 Overall Technical Assessment

The implementation is technically coherent and demonstrates a solid assignment-level software pattern. It separates data validation, preprocessing, model construction, optimizer selection, final training, evaluation, and artifact generation into understandable stages. The preprocessing pipeline is careful about shape normalization and standardization leakage, while the training stage uses a reasonable fully connected architecture with normalization and dropout.

The strongest research-oriented feature is the explicit separation of model development from final test evaluation. The optimizer comparison is conducted before test-time measurement, and the final model is retrained from scratch using the selected configuration. The source also treats diagnostics as first-class artifacts rather than relying on one scalar metric.

The main weaknesses are not coding failures but methodological scope. The model is intentionally simple, the optimizer search is narrow, final validation handling is less controlled than the search split, and the execution environment is only partially pinned. For the stated assignment objective, those choices are reasonable. For a research benchmark, they would need to be tightened and accompanied by a broader experimental protocol.

16 Conclusion

The uploaded project implements a complete supervised deep-learning workflow for Persian handwritten digit recognition using a fully connected neural network. The code reconstructs the serialized HODA samples, normalizes their geometry and intensity range, standardizes features using training-derived statistics, evaluates a compact neural architecture under multiple optimizer settings, retrains a fresh model with adaptive stopping, evaluates the untouched test set, performs class-level diagnostics, inspects a per-sample probability distribution, and packages reproducibility artifacts.

From a software-engineering perspective, the project benefits from explicit validation, modular helper functions, predictable filesystem conventions, structured outputs, and a clear execution path. From a machine-learning perspective, it demonstrates the essential pieces of a controlled supervised-learning experiment: a defined hypothesis class, a training objective, a validation protocol, hyperparameter comparison, a held-out test set, and diagnostic evaluation. The main interpretive point is that the code should be described exactly as implemented: the final training stage passes all 10,000 training examples to Keras but reserves 15% internally for validation during each epoch, and the source computes actual performance metrics at runtime rather than hard-coding them. These distinctions preserve the report's technical fidelity and make the project easier to audit and reproduce.

Source-to-Report Traceability Note

The analysis above was written directly from the uploaded Python source. Key source regions are: environment and seeds (lines 27–67), dataset loading and validation (89–142), preprocessing (144–226), model and optimizer definitions (228–275), hyperparameter search (280–351), final training and training diagnostics (353–427), test prediction diagnostics and probability inspection (428–505), and result export/reproducibility packaging (543–573). The helper-function definitions are located at lines 124–134, 151–163, 165–175, and 241–275.